UNIVERSITY OF ABERDEEN

JC4004 – Computational Intelligence

Sessions 41 & 42: Diffusion models

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

Oct-Dec 2025

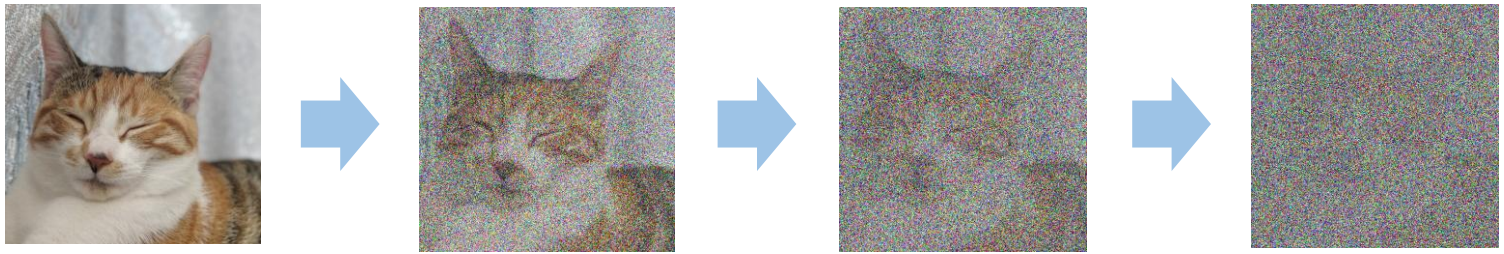
What are diffusion models?

- *Diffusion models* are a class of latent variable generative models
  - Mostly used for image generation, but can be used to generate other types of contents like text, video, and audio as well
- The basic idea of diffusion models is to reverse a process of adding noise to an image to obtain an image with the original characteristics
  - The diffusion models are inspired by physics, especially thermodynamics
  - Diffusion models are trained by adding noise to images and then learning to reverse the process
  - The target images will follow distributions similar with the distributions of the original training images

Forward diffusion process

- The forward diffusion process has a clean image x_0 at time step 0 as a starting point, and at each time step, Gaussian noise is added
  - We denote probability distribution of a clean image as: $x_0 \sim q$
  - At each time step t, Gaussian noise with variance β_t is added, until step T:

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}z_t, \text{ where } z_1, \ldots, z_t \sim \mathcal{N}(0, I)$$

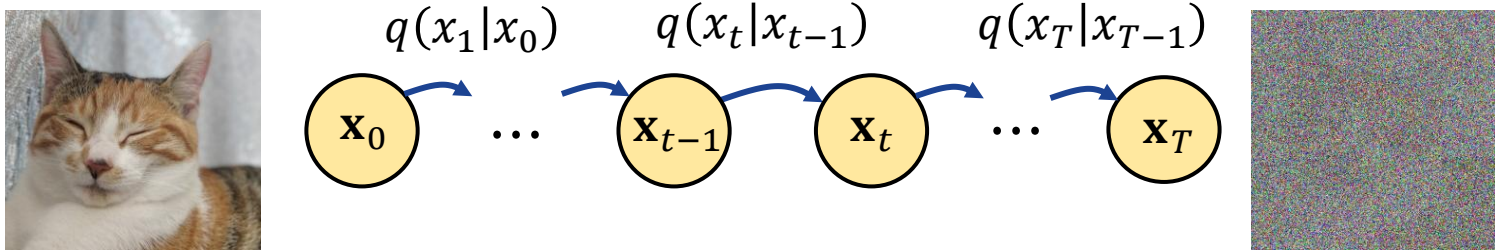


x_0 x_{50} x_{200} x_T

Parametrising diffusion process

- The diffusion process can be considered as a *Markov chain*
- The entire diffusion process satisfies equation:

$$q(x_{0:T}) = q(x_0)q(x_1|x_0)\cdots q(x_T|x_{T-1})$$
$$= q(x_0)\mathcal{N}(x_1|\sqrt{\alpha_1}x_0, \beta_1 I)\cdots \mathcal{N}(x_T|\sqrt{\alpha_T}x_{T-1}, \beta_T I)$$

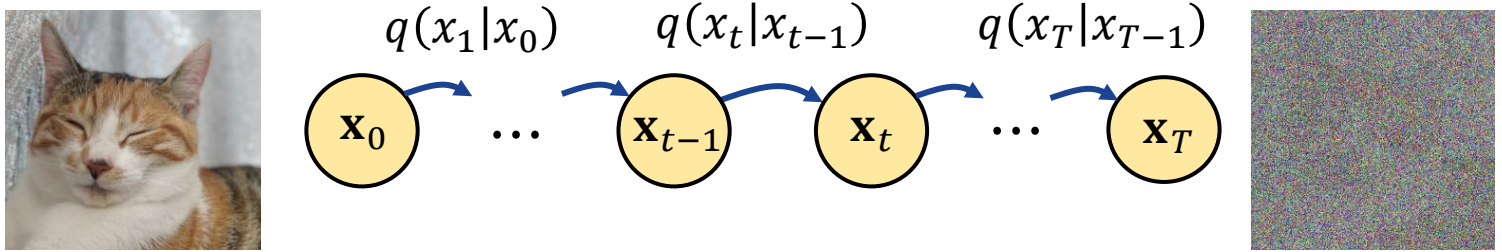


Parametrising diffusion process

- The diffusion process can be considered as a *Markov chain*
- The entire dif

Note that $1 - \beta_t$ is substituted with α_t

$$q(x_{0:T}) = q(x_0)q(x_1|x_0)\cdots q(x_T|x_{T-1})$$
$$= q(x_0)\mathcal{N}(x_1|\sqrt{\alpha_1}x_0, \beta_1 I)\cdots \mathcal{N}(x_T|\sqrt{\alpha_T}x_{T-1}, \beta_T I)$$



Reparametrisation trick

- By substituting $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=0}^{t} \alpha_s$, and given the noise at each time step $\epsilon \sim \mathcal{N}(0, I)$, we can show that:

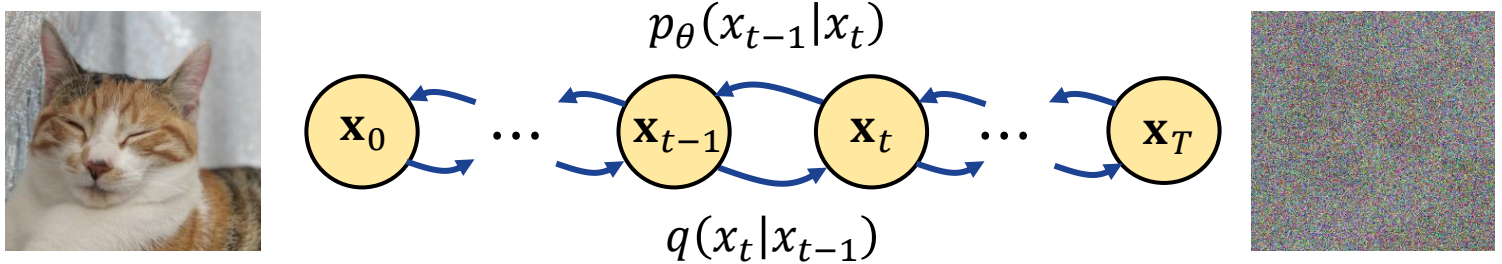
$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t}\epsilon = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t}\epsilon$$

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon$$

- Therefore, we can assume that $x_t | x_0 \sim \mathcal{N}\left(\sqrt{\bar{\alpha}_t} x_0, \sigma_t^2 I\right)$
- For large t, distribution converges towards $x_t | x_0 \sim \mathcal{N}(0, I)$

Reversed diffusion process

- If the variance schedule β_t is properly chosen and there are enough time steps, the final image $x_T \sim \mathcal{N}(0,1)$
  - The interesting question is: *how can we reverse the diffusion process to generate an image from Gaussian noise?*
  - For the reverse process, we can denote that the conditional probability of state x_{t-1} is $p_\theta(x_{t-1}|x_t)$, where θ denotes the model parameters



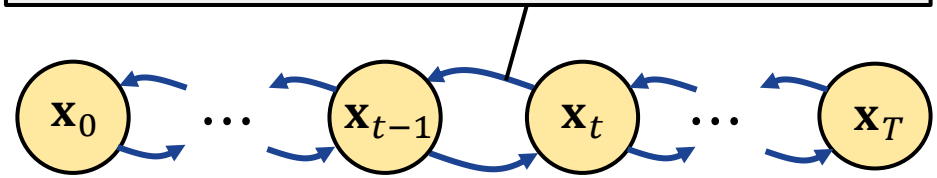
Reverse transition distributions

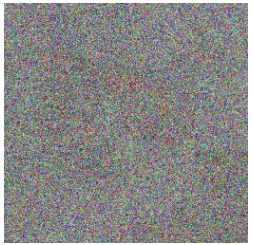
- Starting from pure Gaussian noise $p(x_T) \sim \mathcal{N}(0, I)$, the model is expected to learn the joint distribution $p_\theta(x_{0:T})$:

$$p_\theta(x_{0:T}) = p(x_T)p_\theta(x_{T-1}|x_T) \cdots p_\theta(x_0|x_{1t})$$
$$= \mathcal{N}(0, I)\mathcal{N}\left(x_{T-1}|\mu_\theta(x_T, T), \Sigma_\theta(x_T, T)\right) \cdots \mathcal{N}\left(x_0|\mu_\theta(x_1, 1), \Sigma_\theta(x_1, 1)\right)$$

- Note that transition distribution depends on the previous state only:

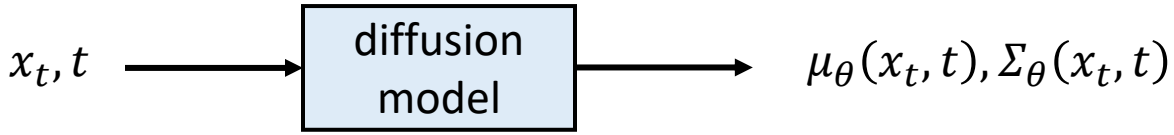
$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1}|\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)\right)$$





Training the model

- The goal of the learning process is to find parameters $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ so that $p_\theta(x_0)$ will be as close to $q(x_0)$ as possible



  - The evidence lower bound (ELBO) inequality states that:

$$E_{x_0 \sim q}[\ln p_\theta(x_0)] \geq E_{x_{0:T} \sim q}[\ln p_\theta(x_{0:T}) - \ln q(x_{1:T}|x_0)]$$

  - Therefore, we can express the loss function $L(\theta)$ to minimise as:

$$L(\theta) = -E_{x_{0:T} \sim q}[\ln p_\theta(x_{0:T}) - \ln q(x_{1:T}|x_0)]$$

Loss function formulation

- The loss function can be expressed in the simplified form:

$$L(\theta) = \sum_{t=1}^{T} E_{x_{t-1},x_t\sim q}[-\ln p_\theta(x_{t-1}|x_t)] - E_{x_0\sim q}[D_{KL}(q(x_T|x_0)\|p(x_T))] + C$$

- Since the second and third terms do not include learnable parameters, the loss function can be further simplified as:

$$L(\theta) = \sum_{t=1}^{T} E_{x_{t-1},x_t\sim q}[-\ln p_\theta(x_{t-1}|x_t)]$$

Noise prediction

- The model must estimate x_0 to predict $\mu_\theta(x_t, t) = \tilde{\mu}_t(x_t, x_0)$
  - Since $x_t|x_0 \sim \mathcal{N}\left(\sqrt{\bar{\alpha}_t}x_0, \sigma_t^2 I\right)$, we can write $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sigma_t\epsilon$, where ϵ is some unknown Gaussian noise
  - Estimating x_0 is equivalent to estimating noise component ϵ: we can train a neural network to estimate $\epsilon_\theta(x_t, t)$, and then predict $\mu_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) = \tilde{\mu}_t\left(x_t, \frac{x_t - \sigma_t\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}\right) = \frac{x_t - \epsilon_\theta(x_t, t)\beta_t/\sigma_t}{\sqrt{\alpha_t}}$$

  - For Σ_θ, we can use fixed value $\Sigma_\theta(x_t, t) = \zeta_t^2 I$, where $\zeta_t^2 = \beta_t$ or $\tilde{\sigma}_t^2$

Loss function for noise prediction

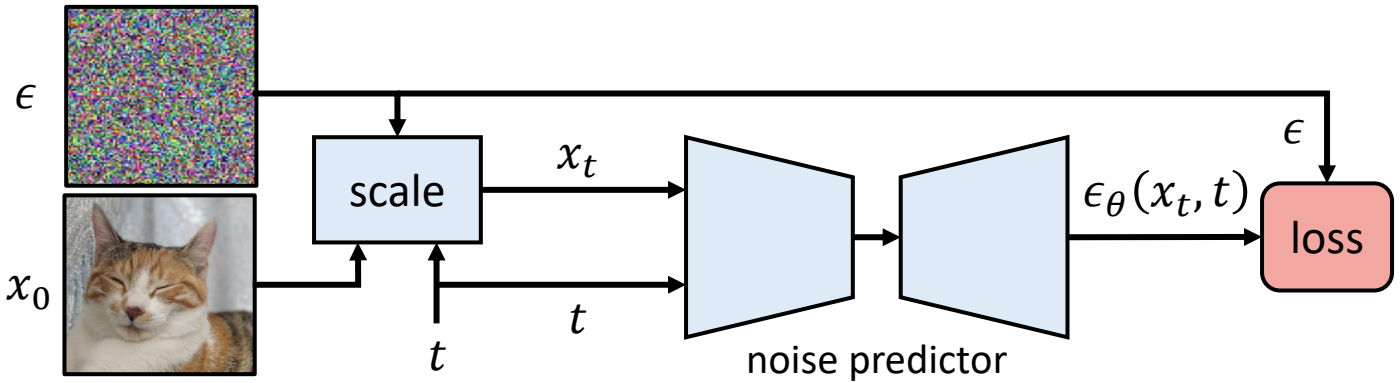
- When training the model for noise prediction, simple squared error between the predicted and actual noise can be used as loss

$$L(\theta) = \sum_{t=1}^{T} E_{x_0 \sim q; z \sim \mathcal{N}(0,I)}[\|\epsilon_\theta(x_t, t) - \epsilon\|^2]$$

  - Experimental results have shown that noise prediction leads to more stable training and better results than predicting $\mu_\theta(x_t, t)$ directly
  - Noise prediction model can be trained by adding noise to clean images and using the loss function above to optimise a neural network

Training a noise predictor

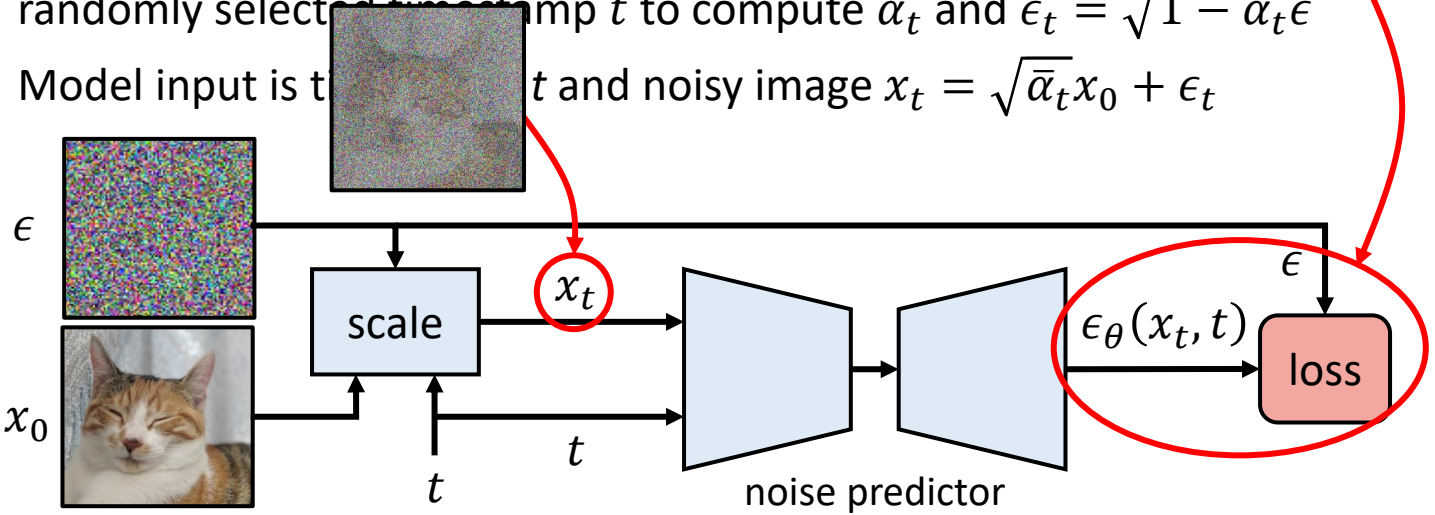
- Most commonly, U-Nets are used for noise prediction
  - In the training process, random noise image $\epsilon \sim \mathcal{N}(0, I)$ is scaled using a randomly selected timestamp t to compute $\bar{\alpha}_t$ and $\epsilon_t = \sqrt{1 - \bar{\alpha}_t}\epsilon$
  - Model input is timestamp t and noisy image $x_t = \sqrt{\bar{\alpha}_t}x_0 + \epsilon_t$



Training a noise predictor

- Most commonly, U-Nets are used for nois
  - In the training process, random noise ima
    randomly select mp t to compute $\bar{\alpha}_t$ and $\epsilon_t = \sqrt{1-\bar{\alpha}_t}\epsilon$
  - Model input is t t and noisy image $x_t = \sqrt{\bar{\alpha}_t}x_0 + \epsilon_t$

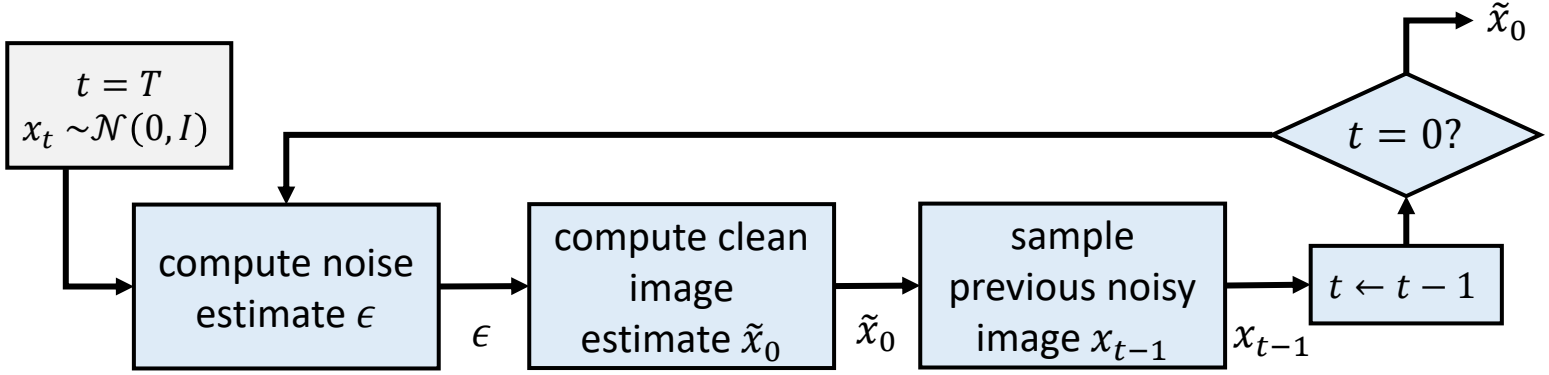
Note that the model aims to predict ϵ, not ϵ_t!



Break: any questions?

Backward diffusion in a trained model

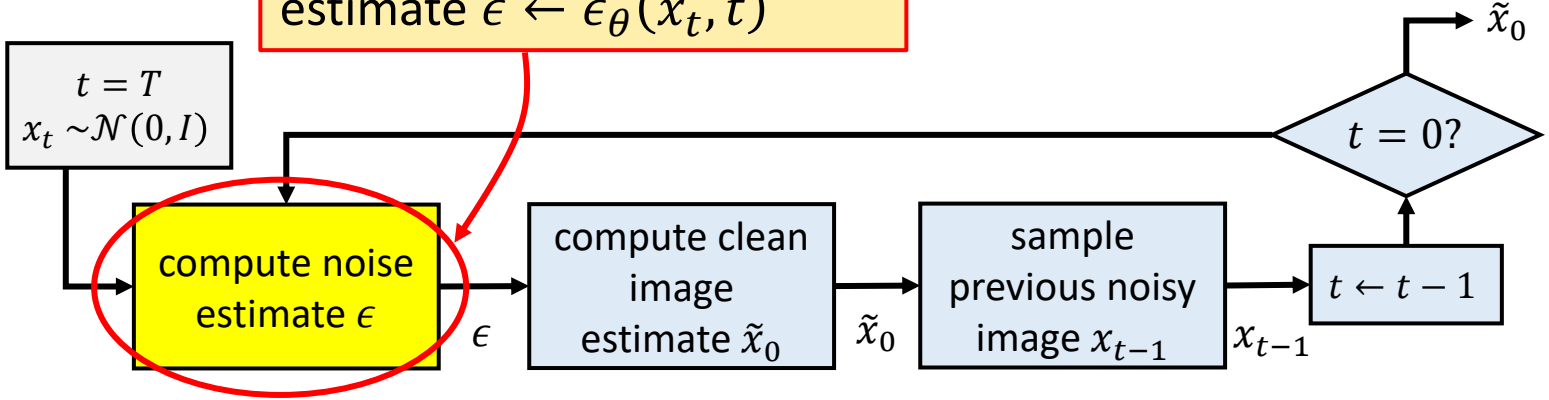
- In the trained model, the original image is estimated at each step
  - The estimated clean image $\tilde{x}_0$ is used to generate the next noisy image
  - We expect $\tilde{x}_0$ to become more and more accurate at each time step



Backward diffusion in a trained model

- In the trained model, the original image is estimated at each step
  - The es… generate the next noisy image
  - We ex… e accurate at each time step

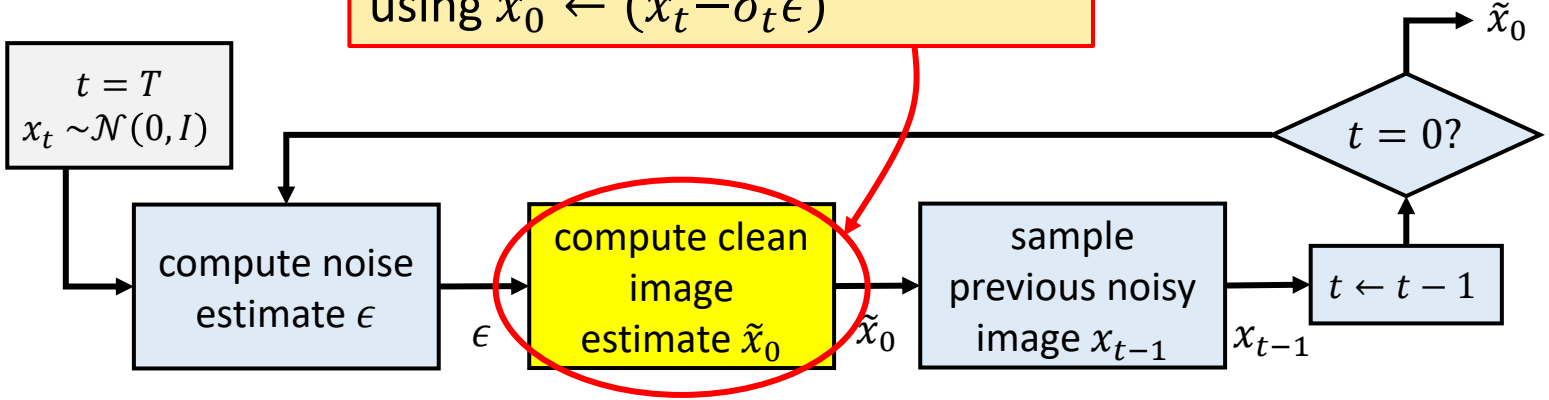
The noise prediction model used to update noise estimate $\epsilon \leftarrow \epsilon_\theta(x_t, t)$



Backward diffusion in a trained model

- In the trained model, the original image is estimated at each step
  - The estimated clean image $\tilde{x}_0$ is used to generate the next noisy image
  - We expect e at each time step

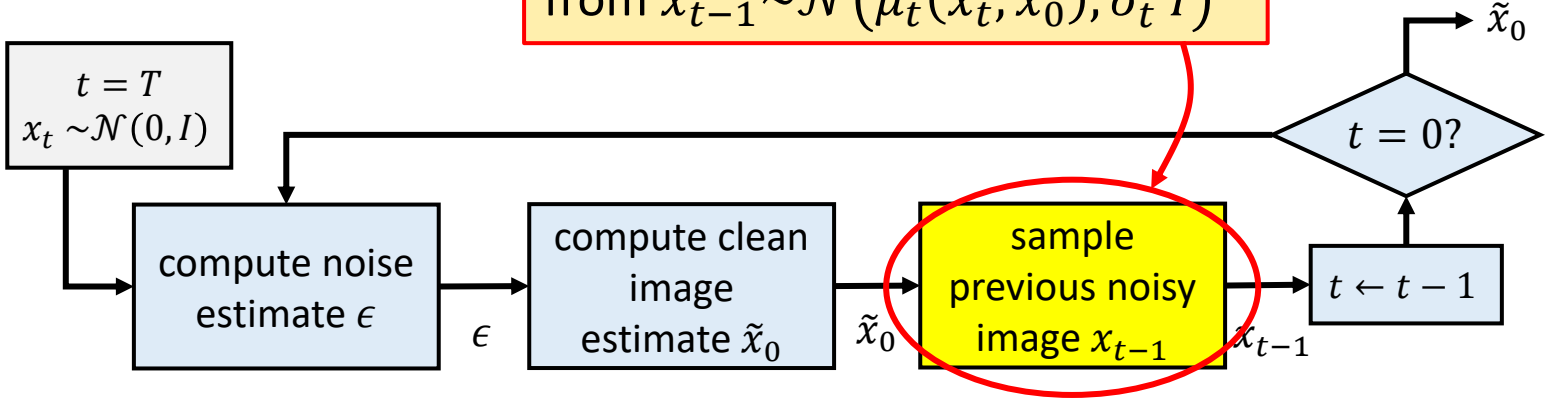
Original clean image estimated using $\tilde{x}_0 \leftarrow (x_t - \sigma_t \epsilon)$



Backward diffusion in a trained model

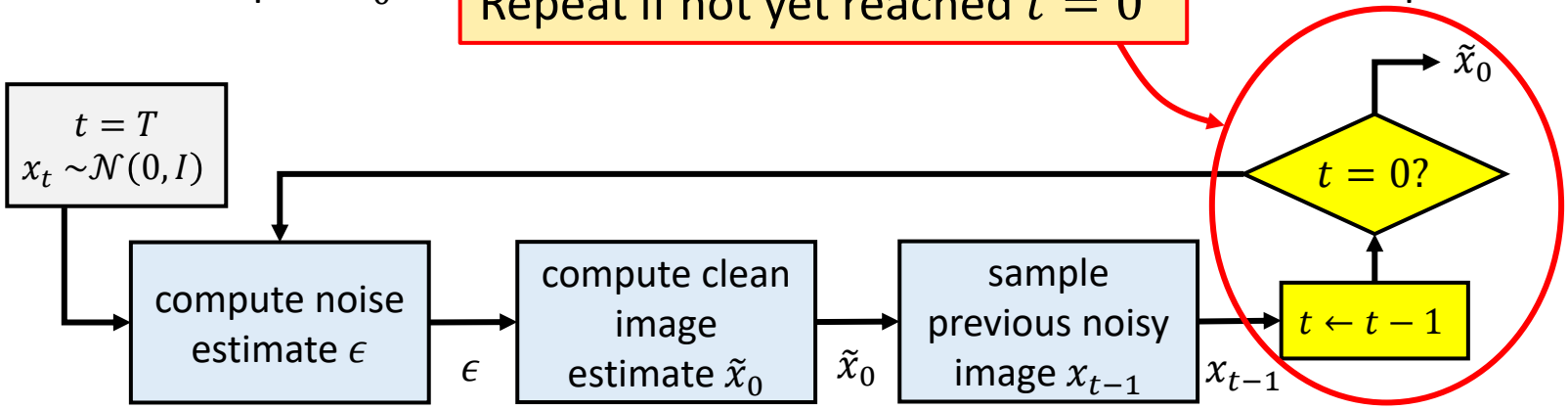
- In the trained model, the original image is estimated at each step
  - The estimated clean image $\tilde{x}_0$ is used to generate the next noisy image
  - We expect $\tilde{x}_0$ to be[illegible] time step

The previous image sampled from $x_{t-1} \sim \mathcal{N}\left(\tilde{\mu}_t(x_t, \tilde{x}_0), \sigma_t^2 I\right)$



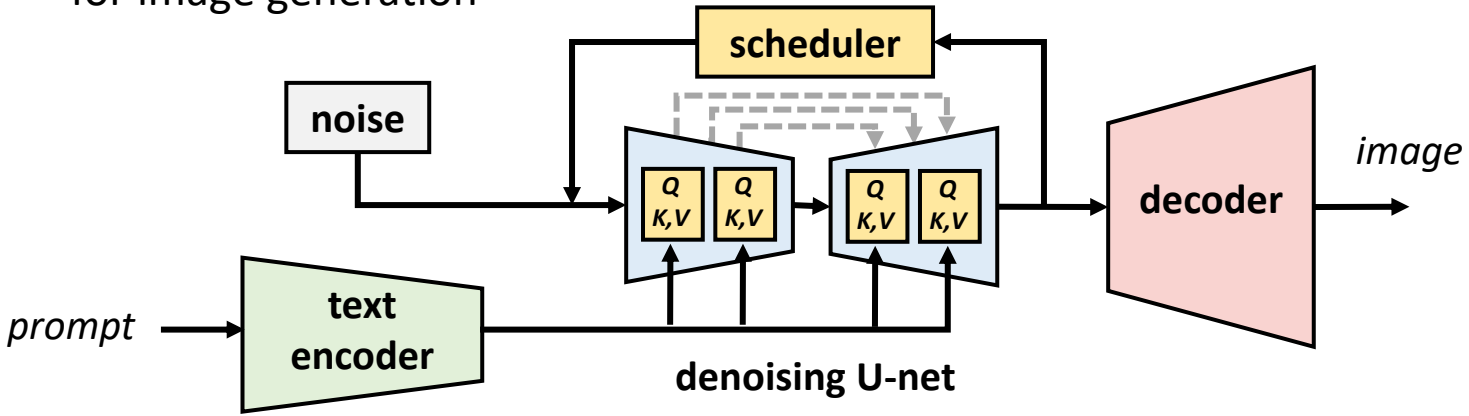
Backward diffusion in a trained model

- In the trained model, the original image is estimated at each step
  - The estimated clean image $\tilde{x}_0$ is used to generate the next noisy image
  - We expect $\tilde{x}_0$ to ch time step



Stable Diffusion

- Practical diffusion models typically combine prompt conditioning with the diffusion generator model
  - *Stable diffusion* is the one of the most successful diffusion models for image generation

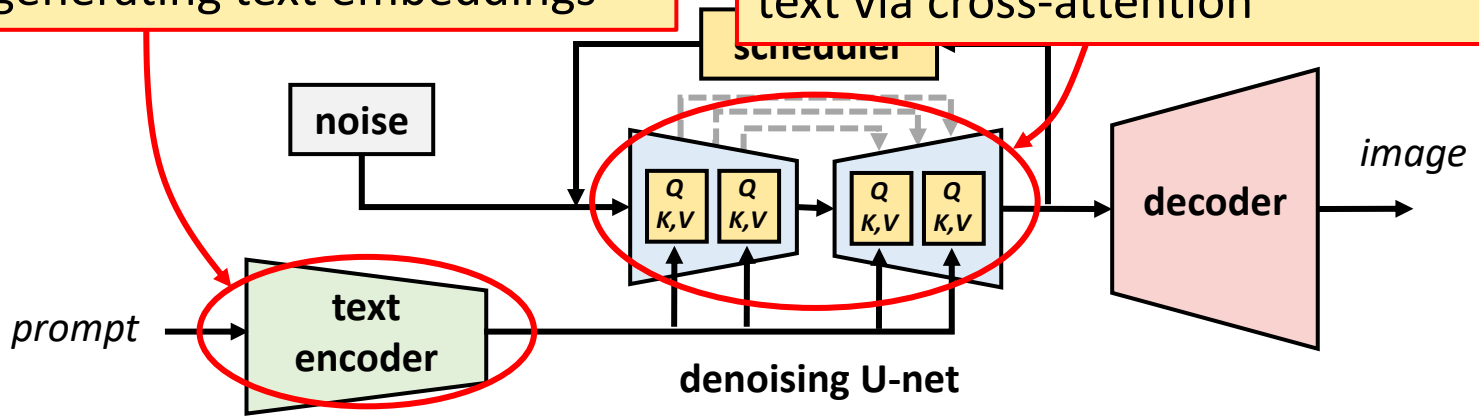


Stable Diffusion

- Practical diffusion models typically
  with the diffusion generator mode

CLIP-based text encoder for generating text embeddings

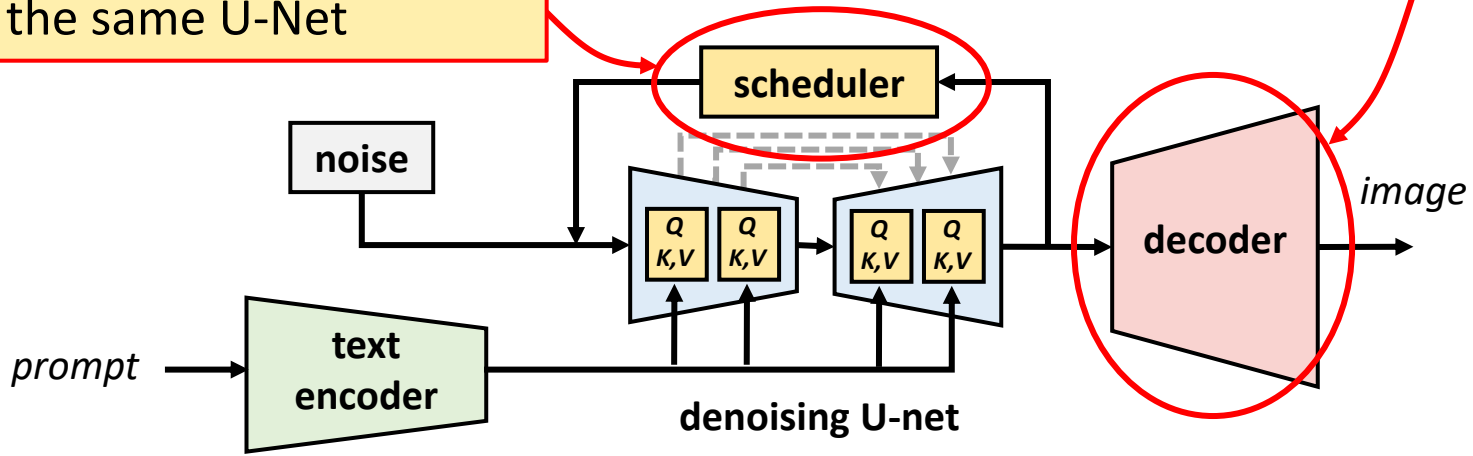
U-Net architecture originally based on ResNet, in later versions on transformers, conditioned to text via cross-attention



Stable Diffusion

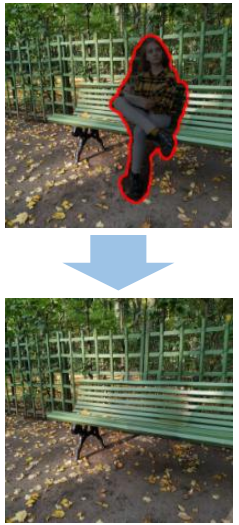
- Refiner schedules several iterations of image refining, using the same U-Net

Decoder generates image from the denoised latent representation; in training, respective encoder is also used



Other Stable Diffusion applications

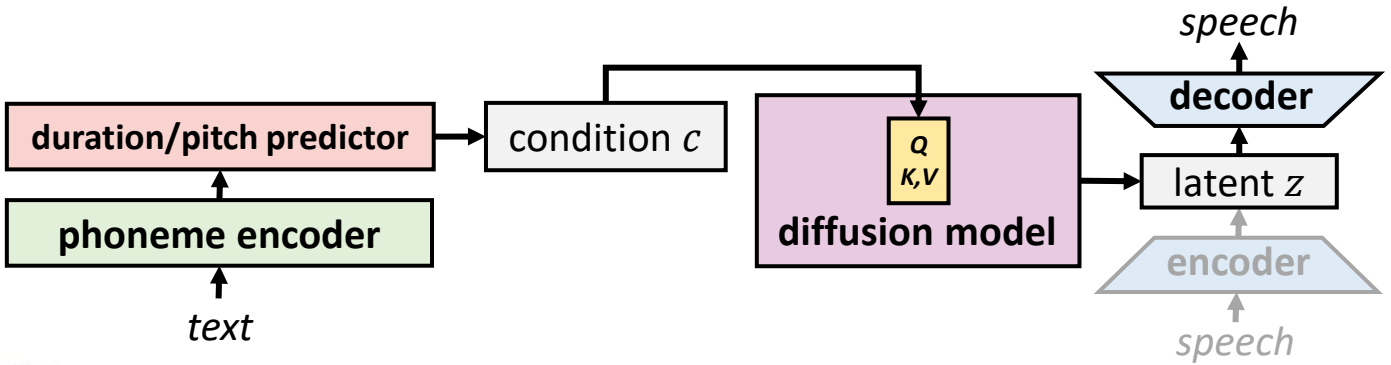
- By using different conditioning techniques and encoders, Stable Diffusion can be adapted for many applications, such as:
  - *Image super-resolution:* generating accurate high-resolution images using a low-resolution image as input
  - *Image modification:* new version of the input image can be generated as instructed by the text prompt
  - *Image inpainting:* masked area of the input image can be repainted, e.g., to remove undesired objects



Source of the image: Rombach et al.: *"High-Resolution Image Synthesis with Latent Diffusion Models,"* CVPR 2022.

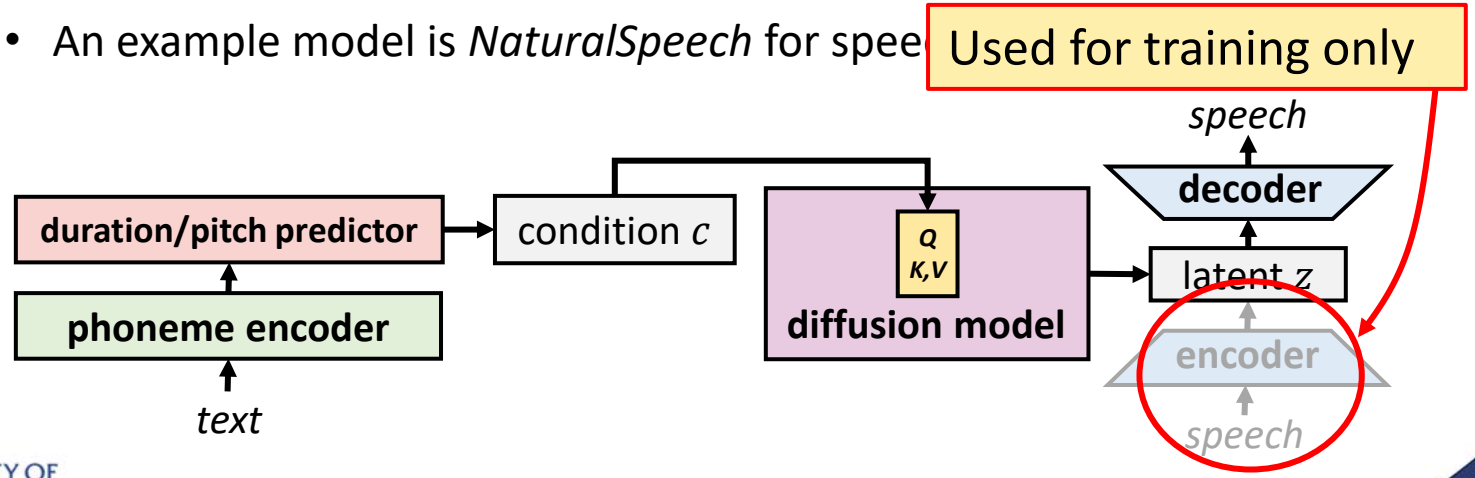
Diffusion models for audio

- Diffusion models can be also used for audio generation, including text-to-speech and music generation
  - The basic idea is similar with image generation, although there are some differences in details such as U-Net model architectures
  - An example model is *NaturalSpeech* for speech and singing synthesis



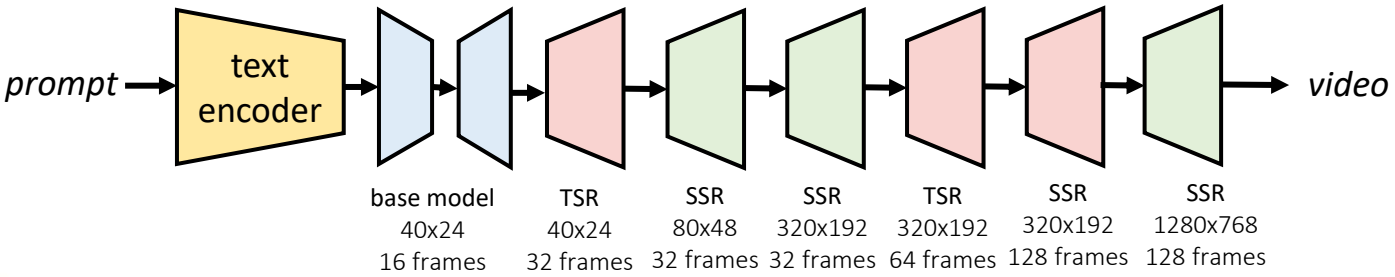
Diffusion models for audio

- Diffusion models can be also used for audio generation, including text-to-speech and music generation
  - The basic idea is similar with image generation, although there are some differences in details such as U-Net model architectures
  - An example model is *NaturalSpeech* for spee



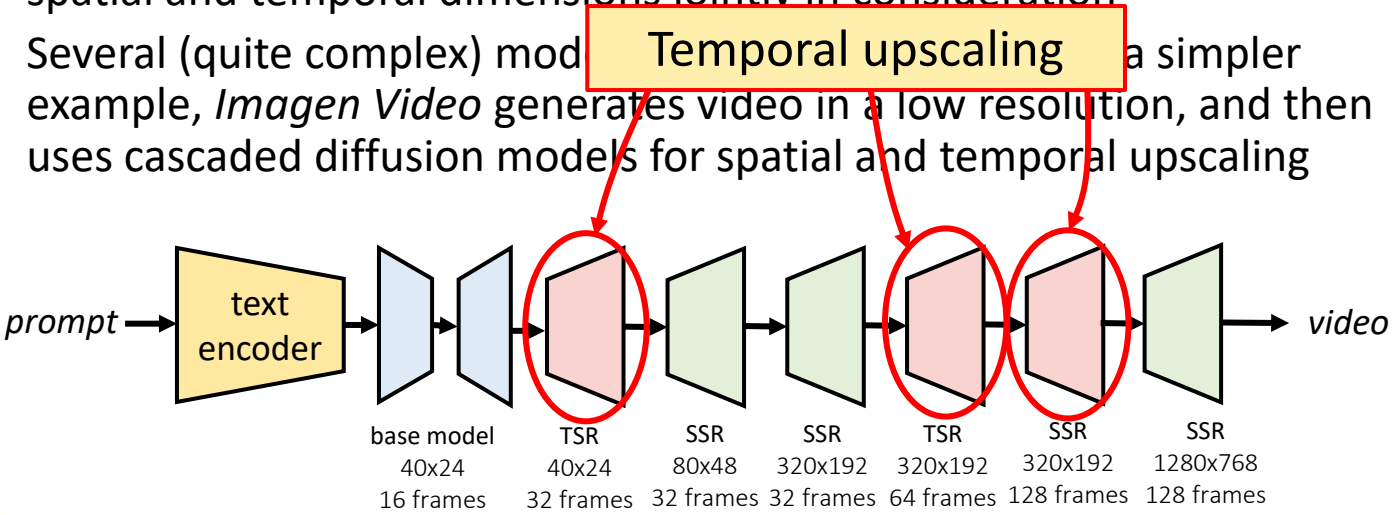
Diffusion models for video

- Diffusion models for video generation have also been developed
  - The challenge is to achieve smooth motion: the model must take spatial and temporal dimensions jointly in consideration
  - Several (quite complex) models have been proposed; as a simpler example, *Imagen Video* generates video in a low resolution, and then uses cascaded diffusion models for spatial and temporal upscaling



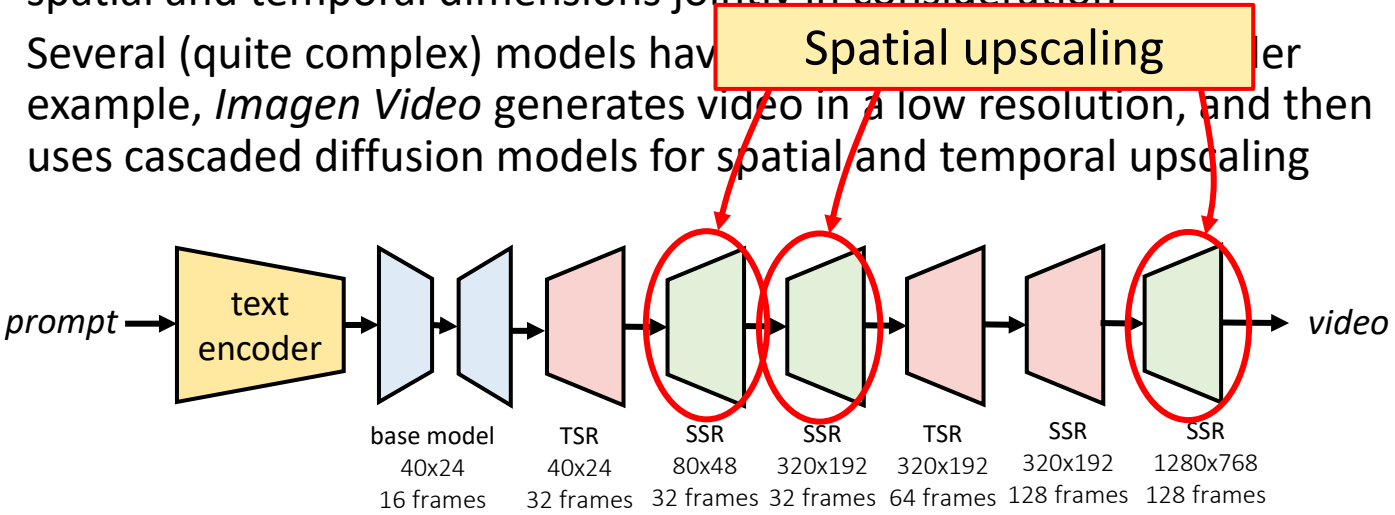
Diffusion models for video

- Diffusion models for video generation have also been developed
  - The challenge is to achieve smooth motion: the model must take spatial and temporal dimensions jointly in consideration
  - Several (quite complex) mod… a simpler example, *Imagen Video* generates video in a low resolution, and then uses cascaded diffusion models for spatial and temporal upscaling



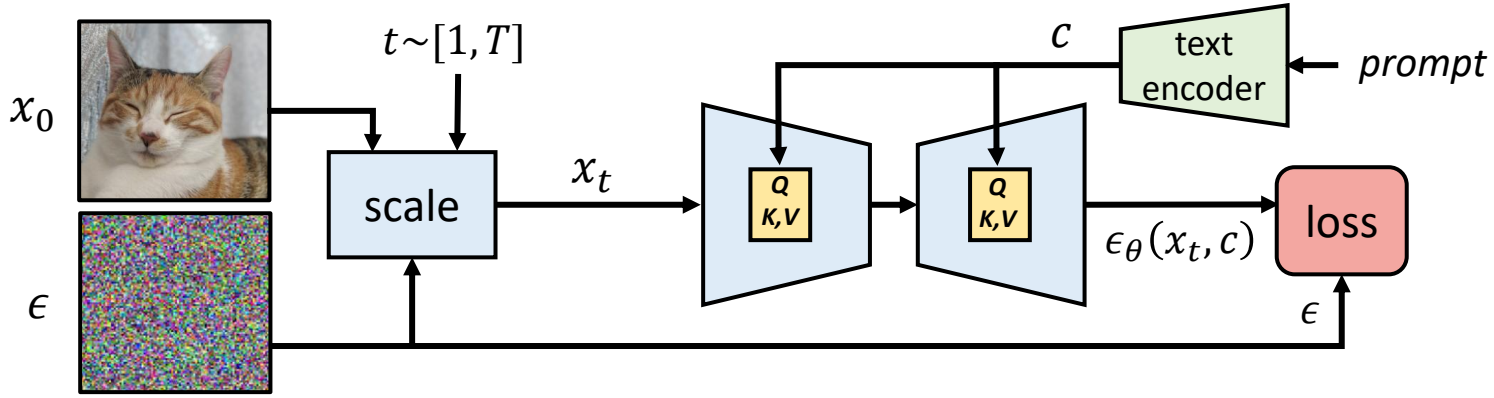
Diffusion models for video

- Diffusion models for video generation have also been developed
  - The challenge is to achieve smooth motion: the model must take spatial and temporal dimensions jointly in consideration
  - Several (quite complex) models hav ler example, *Imagen Video* generates video in a low resolution, and then uses cascaded diffusion models for spatial and temporal upscaling



Diffusion classifier

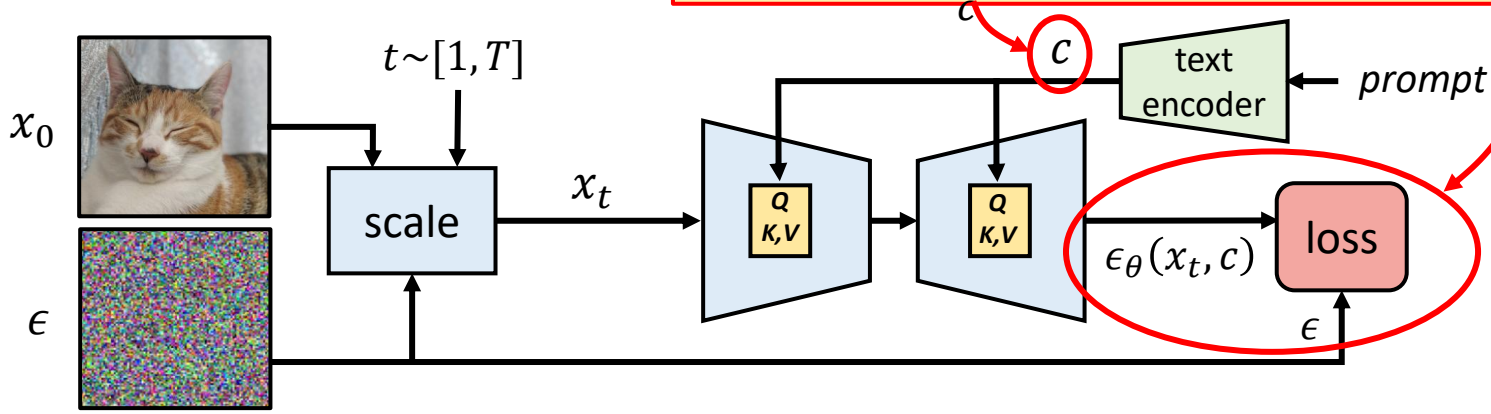
- Diffusion models can also be used for zero-shot (not fine-tuned) classification and compositional reasoning tasks
  - The basic idea is to find a text prompt that minimises the difference between real noise and noise predicted by conditioned noise predictor



Diffusion classifier

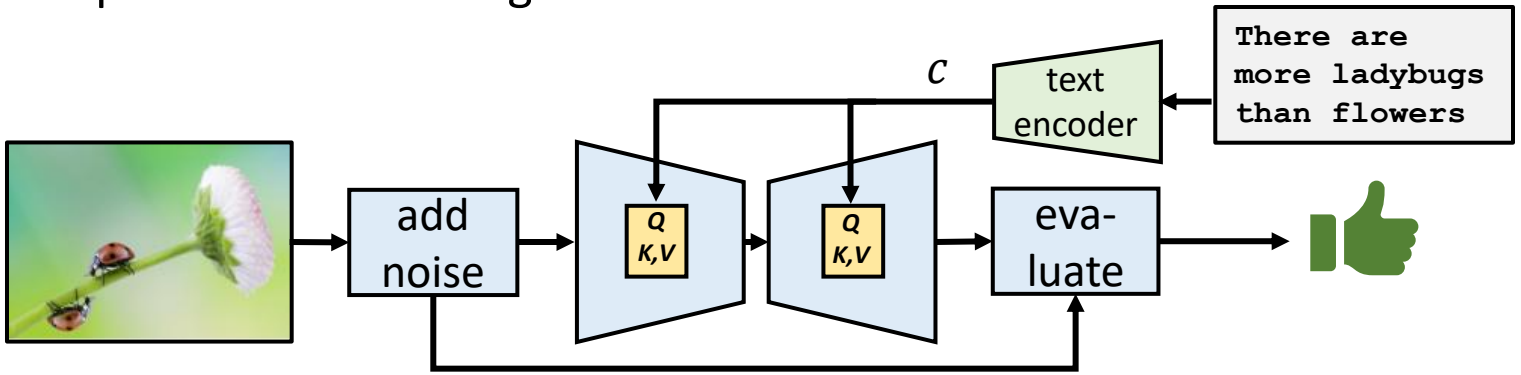
- Diffusion models can also be used for zero-shot (not fine-tuned) classification and compositional reasoning tasks
  - The basic idea is to find a text
    between real noise and noise

The objective is to find c that minimises the loss: $\text{argmin}\left(E_{t,\epsilon}[\|\epsilon_\theta(x_t, c) - \epsilon\|^2]\right)$



Diffusion classifier advantages

- Diffusion classifier achieves competitive performance in zero-shot classification, but it does not clearly outperform CLIP based models
- However, diffusion classifier shows an advantage in more complex compositional reasoning tasks



Summary

- *Diffusion models* are a recently developed class of generative AI models originally designed for image generation
  - The basic idea is to learn the reverse process of gradually adding Gaussian noise to clean training content
  - The content generation process can be conditioned to e.g., text prompts
- Diffusion models have a wide range of practical applications
  - For example, prominent *Stable Diffusion* model can be used for image super-resolution and inpainting, in addition to image synthesis
  - Diffusion models can be used also for audio and video generation
  - *Diffusion classifiers* use pre-trained diffusion models for zero-shot classification and

References

- Ho et al.: *“Denoising Diffusion Models,”* NeurIPS 2020.
- https://en.wikipedia.org/wiki/Diffusion_model
- Rombach et al.: *“High-Resolution Image Synthesis with Latent Diffusion Models,”* CVPR 2022.
- Ho et al.: *“Imagen Video: High Definition Video Generation with Diffusion Models,”* arXiv.2210.02303, 2022.
- Li et al.: *“Your Diffusion Model is Secretly a Zero-Shot Classifier,”* ICCV 2023.
- Shen et al.: *“NaturalSpeech 2: Latent Diffusion Models are Naturaland Zero-Shot Speech and Singing Synthesizers,”* ICLR 2024.

Questions and discussion